Batched Backpropagation as a Linear Solve

Mikkel Paltorp (mpasc@dtu.dk), 7 October 2026
Department of Applied Mathematics and Computer Science, Technical University of Denmark

This post extends Backpropagation as a Linear Solve[1] to batches of inputs. The main change is that the (sparse) matrices of scalars from the previous post become (sparse) matrices whose entries are linear maps.[2] To readers used to the usual presentation of backpropagation as a pass over a computational graph, matrices may seem an odd choice. The two views are the same: the sparsity pattern below the diagonal of a lower triangular matrix $\mathcal{L}$ is a Directed Acyclic Graph (DAG).

The post ends with a Julia implementation of a neural network whose gradient is $\nabla_\theta \mathcal{J} = \mathcal{M}\left((\mathcal{L}^*)^{-1}\nabla_U \ell\right) + \nabla_\theta \ell$, the same formula as before, written as a linear solve in the style of Edelman, Akyürek, and Wang [1]. The inverse is never formed: because $\mathcal{L}$ is unit lower triangular, $\mathcal{L}^*$ is unit upper triangular, and its inverse can be applied efficiently using back substitution.

Contents

1. The batched forward pass
2. Generalization to vector spaces
   2.1. The loss
   2.2. Adjoints
   2.3. Gradients
3. The batched gradient
   3.1. The forward relation
   3.2. The adjoint equation
   3.3. The operators as block (sparse) matrices
4. A code example
5. Conclusion
Bibliography

[1] Earlier post: *Backpropagation as a Linear Solve*. mipals.github.io/blog/2026/10-05-backpropagation-linear-solve/

[2] More often than not the matrices are *sparse*—which is key for the efficiency of automatic differentiation.

1. The batched forward pass

The previous post considered a single input u_0 and a loss $\ell(u_P)$ of its outputs. However, the loss function is often based on minimizing a data-fitting term of a *minibatch* of m input-output pairs $(u_0^{(s)}, y^{(s)})$. Due to the linearity of a sum the full gradient over the m input-output data could be computed by simply summing the gradient contribution for each example. In practice, it is often more efficient to process the data together in a batch.[3] The first step is to introduce the batched input and output matrices by stacking the data points as columns, i.e.

$$U_0 = \begin{pmatrix} u_0^{(1)} & \dots & u_0^{(m)} \end{pmatrix}, \quad Y = \begin{pmatrix} y^{(1)} & \dots & y^{(m)} \end{pmatrix}. \tag{1}$$

The only difference between this and the single input case is that the application of layer i now generates m outputs. Writing the outputs with a capital U, as for the input data, layer i gives

$$U_i = \Phi_i(U_{i-1}, \dots, U_0, \theta). \tag{2}$$

We denote the collection of all the states by $U(\theta)$ and the collection of all the layer maps by $\Phi(U, \theta)$. One can think of them as *vectors* whose entries are the states and the layer maps, respectively, so that

$$\begin{aligned} U(\theta) &= (U_1, U_2, \dots, U_N), \\ \Phi(U, \theta) &= (\Phi_1(U_0, \theta), \Phi_2(U_1, U_0, \theta), \dots, \Phi_N(U_{N-1}, \dots, U_0, \theta)). \end{aligned} \tag{3}$$

However, this does not have to be the case, and the only thing we require is that $U(\theta)$ is an element of a vector space $\mathcal{U}$ with an inner product. Similarly, the parameters θ live in a vector space Θ. The next section makes this precise.[4]

[3] Batching turns matrix–vector products into matrix–matrix products, which use modern hardware far better. Moreover, because the weights are shared by all the data points, they only have to be loaded into memory once for a batch, whereas processing each example separately would load them once per example.

[4] See Wikipedia for what is meant by a vector space.

2. Generalization to vector spaces

It is standard to equip the space that each layer output lives in with the Frobenius inner product. This means that for $Z, E \in \mathcal{U}$ the inner product is a sum of Frobenius inner products over the layers, i.e.

$$\langle Z, E\rangle_{\mathcal{U}} = \sum_{i=1}^{N} \langle Z_i, E_i\rangle_F = \sum_{i=1}^{N} \operatorname{tr}(Z_i^\top E_i). \tag{4}$$

The construction of the parameter space is slightly more complicated, as parameters can be shared between layers.[5] The parameters can still be thought of as a vector whose entries are *parameter blocks*, that is, $\theta = (\theta_k)_{k=1}^{K} \in \Theta$. Layer i uses only some of the blocks, and several layers may use the same block. A linear selection map $\mathcal{S}_i : \Theta \to \Theta_{\mathcal{S}_i}$ picks out the blocks of layer i, so that Φ_i depends on θ only through $\theta_{\mathcal{S}_i} = \mathcal{S}_i(\theta)$. In a plain feed-forward network every layer has its own weight block $W_{i,i-1}$ and bias block b_i, and $\mathcal{S}_i(\theta) = \big(W_{i,i-1}, b_i\big)$. Similar to $\mathcal{U}$, we equip Θ with the Frobenius inner product block by block, so that for $h, g \in \Theta$ it follows that

[5] For example, in a recurrent neural network.

$$\langle h, g\rangle_\Theta = \sum_{k=1}^{K} \langle h_k, g_k\rangle_F = \sum_{k=1}^{K} \operatorname{tr}\big(h_k^\top g_k\big). \tag{5}$$

Lastly, we also aim to equip the output space $\mathcal{Y}$ with an inner product. We view elements in $\mathcal{Y}$ as a list of outputs, $A = (A_1, \ldots, A_r)$, where again the inner product is a sum of Frobenius inner products over the list,

$$\langle A, B\rangle_{\mathcal{Y}} = \sum_{a=1}^{r} \langle A_a, B_a\rangle_F = \sum_{a=1}^{r} \operatorname{tr}(A_a^\top B_a). \tag{6}$$

We then define a linear output map $\mathcal{P} : \mathcal{U} \to \mathcal{Y}$ such that it maps the full state $U \in \mathcal{U}$ to a corresponding output. That is $\mathcal{P}(U) \in \mathcal{Y}$ and we sometimes simply write $U_{\mathcal{P}} = \mathcal{P}(U)$.[6]

[6] Usually $\mathcal{P}$ picks out the last layer, $\mathcal{P}(U) = U_N$. It can also select several layers, or combine them, for example by averaging.

2.1. The loss

We often describe the loss function as a function of the parameters θ only, but it is more natural to view it as a function of the states and the parameters. That is, $\mathcal{J}(\theta) = \ell(U(\theta), \theta)$, where usually

$$\ell(U, \theta) = f(\mathcal{P}(U); Y) + \mathcal{R}(\theta), \tag{7}$$

where f is a data-fitting term that compares the outputs with the targets Y, and $\mathcal{R}$ is a regularization term. The derivation below only needs ℓ and Φ to be differentiable, which also makes $\mathcal{J}$ differentiable. Only the gradients $\nabla_U \ell$ and $\nabla_\theta \ell$ of ℓ enter, as the right-hand side of the adjoint equation and as the last term of the gradient.

Example 1: Squared loss

Example 1 (Squared loss) The natural extension of the scalar loss is the squared Frobenius norm of the residuals, written with the inner product on $\mathcal{Y}$,[7] together with a regularization of the parameters,

$$f(A; Y) = \frac{1}{2m}\langle A - Y, A - Y\rangle_{\mathcal{Y}}, \quad \mathcal{R}(\theta) = \frac{\alpha}{2}\langle \theta, \theta\rangle_\Theta. \tag{8}$$

This is the loss of the examples and the code of this post.

[7] The scalar loss comes from the inner product

$$\langle u - y, u - y\rangle = (u - y)^\top (u - y).$$

2.2. Adjoints

For a linear map $\mathcal{A} : \mathcal{X} \to \mathcal{Z}$ between two of these spaces, the adjoint $\mathcal{A}^* : \mathcal{Z} \to \mathcal{X}$ is defined by

$$\langle \mathcal{A}(x), z\rangle_{\mathcal{Z}} = \langle x, \mathcal{A}^*(z)\rangle_{\mathcal{X}} \quad \text{for all} \quad x \in \mathcal{X}, z \in \mathcal{Z}. \tag{9}$$

Moving an operator across an inner product therefore changes the inner product from the operator's output space to its input space.

Example 2: Adjoint of the selection map

Example 2 (Adjoint of the selection map) Take $N = 3$ layers, each producing a matrix U_i with m columns, one per data point, and let the loss depend on the first and the last layer output. Then

$$U = (U_1, U_2, U_3) \in \mathcal{U}, \quad A = (A_1, A_2) \in \mathcal{Y}, \tag{10}$$

where A_1 and A_2 have the sizes of U_1 and U_3. The selection map $\mathcal{P} : \mathcal{U} \to \mathcal{Y}$ is $\mathcal{P}(U) = (U_1, U_3)$. We are free to select other layers, or the same layer more than once, as long as each selected U_i has the size of the corresponding entry of $\mathcal{Y}$. For any $U \in \mathcal{U}$ and $A \in \mathcal{Y}$, writing out the inner product on $\mathcal{Y}$ and adding a zero term for the unselected layer gives

$$\begin{aligned} \langle \mathcal{P}(U), A \rangle_{\mathcal{Y}} &= \langle U_1, A_1 \rangle_F + \langle U_3, A_2 \rangle_F \\ &= \langle U_1, A_1 \rangle_F + \langle U_2, 0 \rangle_F + \langle U_3, A_2 \rangle_F = \langle U, (A_1, 0, A_2) \rangle_{\mathcal{U}} \end{aligned} \tag{11}$$

Hence the adjoint $\mathcal{P}^*$ places the selected blocks back in their original positions, with zeros elsewhere, i.e. $\mathcal{P}^*(A) = (A_1, 0, A_2)$. If a layer is selected more than once, $\mathcal{P}^*$ sums the corresponding blocks of A in its position. The inner product starts in $\mathcal{Y}$, the output space of $\mathcal{P}$, and ends in $\mathcal{U}$, its input space. In the common case where only the last layer is selected, $\mathcal{P}(U) = U_N$ and $\mathcal{P}^*(A) = (0, \ldots, 0, A)$.

2.3. Gradients

The gradient is defined through an inner product. For a differentiable scalar function of θ, such as the loss $\mathcal{J}$, the derivative $D\mathcal{J}(\theta) \in \Theta^*$ is a linear map from directions $h \in \Theta$ to numbers.[8] The Riesz representation theorem[9] says that such a map is an inner product with a fixed element of Θ. That element is the gradient

$$D\mathcal{J}(\theta)[h] = \langle \nabla_\theta \mathcal{J}, h \rangle_\Theta \quad \text{for every direction} \quad h. \tag{12}$$

So to find a gradient, we write the derivative in the form $\langle \cdot, h \rangle_\Theta$ and read off the other argument. The same definition applies on $\mathcal{U}$ and $\mathcal{Y}$, each with its own inner product. Changing the inner product therefore changes the gradient, but not the derivative.

[8] Θ^* denotes the *dual* space of Θ, which is the space of all linear functionals on Θ. This is a technicality that we revisit in *Which Way Is Down? First-Order Optimizers as a Choice of Norm*.

[9] In a finite-dimensional inner product space every linear functional ψ has the form $\psi(h) = \langle g, h \rangle_\Theta$ for exactly one $g \in \Theta$, its Riesz representative.

Example 3: Gradients of the loss

Example 3 (Gradients of the loss) A state change $E \in \mathcal{U}$ changes the outputs by $\mathcal{P}(E)$, since $\mathcal{P}$ is linear. Moving $\mathcal{P}$ across the inner product with its adjoint gives

$$D_U \ell[E] = \langle \nabla f, \mathcal{P}(E) \rangle_{\mathcal{Y}} = \langle \mathcal{P}^*(\nabla f), E \rangle_{\mathcal{U}}, \quad \text{so} \quad \nabla_U \ell = \mathcal{P}^*(\nabla f), \tag{13}$$

with ∇f evaluated at $U_{\mathcal{P}}$. For the squared loss, $\nabla f = (U_{\mathcal{P}} - Y)/m$,[10] and $D_\theta \ell[h] = \alpha \langle \theta, h \rangle_\Theta$ gives $\nabla_\theta \ell = \alpha \theta$.

[10] By the product rule and the symmetry of the inner product, $D\langle a, a \rangle[E] = 2\langle a, Da[E] \rangle$ [2].

The last step depends on the inner product on Θ. With $\langle g, h \rangle_{\mathcal{A}} = \langle \mathcal{A}(g), h \rangle_\Theta$ instead, for a self-adjoint and positive definite $\mathcal{A}$, the same derivative reads

$$D_\theta \ell[h] = \alpha \langle \theta, h \rangle_\Theta = \alpha \langle \mathcal{A}(\mathcal{A}^{-1}(\theta)), h \rangle_\Theta = \langle \alpha \mathcal{A}^{-1}(\theta), h \rangle_{\mathcal{A}}, \tag{14}$$

so the gradient becomes $\alpha\mathcal{A}^{-1}(\theta)$. In the same way, the gradient of $\mathcal{J}$ becomes the preconditioned $\mathcal{A}^{-1}(\nabla_\theta\mathcal{J})$, which Which Way Is Down?[11] builds optimizers from.

[11] Later post: *Which Way Is Down? First-Order Optimizers as a Choice of Norm*. mipals.github.io/blog/2026/10-13-steepest-descent/

3. The batched gradient

Note that the parameters enter the loss function $\mathcal{J}(\theta) = \ell(U(\theta), \theta)$ twice: directly as θ, and through the states $U(\theta)$. Differentiating in a parameter direction $h \in \Theta$, the chain rule gives one term for each,

$$\begin{aligned} D\mathcal{J}(\theta)[h] &= D_U\ell[Z_h] + D_\theta\ell[h] = \langle \nabla_U\ell, Z_h\rangle_{\mathcal{U}} + \langle \nabla_\theta\ell, h\rangle_\Theta, \\ Z_h &= DU(\theta)[h] \in \mathcal{U}. \end{aligned} \tag{15}$$

In order to find the gradient $\nabla_\theta\mathcal{J}$, we need to write the first term in the form $\langle \cdot\,, h\rangle_\Theta$ so that we can read off the other argument—the Riesz representative of the derivative.

3.1. The forward relation

To relate Z_h to h, differentiate the forward relation $U(\theta) = \Phi(U(\theta), \theta)$ in the direction h. The left side gives Z_h. On the right, θ again enters both directly and through $U(\theta)$, and the chain rule gives

$$Z_h = D_U\Phi[Z_h] + D_\theta\Phi[h]. \tag{16}$$

The first term accounts for changes through the states, while the second accounts for the explicit parameter dependence.[12] Moving the state term to the left gives the linear equation

$$\underbrace{(\mathcal{I} - D_U\Phi)}_{\mathcal{L}}(Z_h) = \underbrace{D_\theta\Phi}_{\mathcal{M}^*}[h], \quad \mathcal{L} : \mathcal{U} \to \mathcal{U}, \quad \mathcal{M}^* : \Theta \to \mathcal{U}, \tag{17}$$

where $\mathcal{L}$ and $\mathcal{M}^*$ are the operator versions of L and $M^\top$ from the previous post. Solving it, $\mathcal{L}(Z_h) = \mathcal{M}^*(h)$, gives Z_h for one direction h. This is forward-mode differentiation, and the full gradient this way would need one solve per direction in a basis of Θ, that is, $\dim\Theta$ solves.

[12] Both partial derivatives are evaluated at the current (U, θ). In $D_U\Phi$, θ is held fixed; in $D_\theta\Phi$, U is held fixed. The total state dependence enters through Z_h.

3.2. The adjoint equation

Solving Equation 17 gives $Z_h = \mathcal{L}^{-1}(\mathcal{M}^*(h))$. Substituting this into the first term of Equation 15 and moving first $\mathcal{L}^{-1}$, which maps $\mathcal{U}$ to itself, and then $\mathcal{M}^*$, which maps Θ to $\mathcal{U}$, across the inner product using their respective adjoints gives

$$\langle \nabla_U\ell, \mathcal{L}^{-1}(\mathcal{M}^*(h))\rangle_{\mathcal{U}} = \langle (\mathcal{L}^{-1})^*(\nabla_U\ell), \mathcal{M}^*(h)\rangle_{\mathcal{U}} = \langle \mathcal{M}(\Lambda), h\rangle_\Theta, \tag{18}$$

where $\Lambda = (\mathcal{L}^{-1})^*(\nabla_U\ell)$. Rather than forming the inverse, we compute Λ by solving the adjoint equation[13]

$$\mathcal{L}^*(\Lambda) = \nabla_U\ell, \tag{19}$$

whose right-hand side does not depend on h, so a single solve serves every direction. This is reverse-mode differentiation: as shown below, $\mathcal{L}$ is lower block triangular, so $\mathcal{L}^*$ is upper block triangular, and the solve runs from the last layer back to the first. Collecting the terms,

$$D\mathcal{J}(\theta)[h] = \langle \mathcal{M}(\Lambda) + \nabla_\theta\ell, h\rangle_\Theta \quad \text{for every} \quad h. \tag{20}$$

[13] For invertible $\mathcal{L}$, the inverse and adjoint commute:

$$(\mathcal{L}^{-1})^* = (\mathcal{L}^*)^{-1}.$$

Taking adjoints of $\mathcal{L}^{-1}\mathcal{L} = \mathcal{I}$ reverses the order, giving $\mathcal{L}^*(\mathcal{L}^{-1})^* = \mathcal{I}$; the reverse product is also the identity. Moving $\mathcal{L}^{-1}$ across the inner product therefore gives a solve with $\mathcal{L}^*$.

Reading off the Riesz representative of $D\mathcal{J}(\theta)$ gives

$$\nabla_\theta \mathcal{J} = \mathcal{M}(\Lambda) + \nabla_\theta \ell \in \Theta. \tag{21}$$

3.3. The operators as block (sparse) matrices

Viewing the states and the parameters as lists, $\mathcal{L}$ and $\mathcal{M}^*$ can be written as block matrices whose entries are linear maps. For a state change $E \in \mathcal{U}$, layer i can only respond to the states that come strictly before it, so entry i of $\mathcal{L}(E)$[14] is

$$(\mathcal{L}(E))_i = E_i - \sum_{j=1}^{i-1} D_{U_j}\Phi_i\big[E_j\big] = \sum_{j=1}^{i} \mathcal{L}_{ij}(E_j), \quad \mathcal{L}_{ij} = \begin{cases} \mathcal{I} & \text{if } j = i \\ -D_{U_j}\Phi_i & \text{if } j < i \end{cases} \tag{22}$$

so block (i, j) of $\mathcal{L}$ is nonzero only if layer i reads the state U_j. In the same way, for a parameter direction $h \in \Theta$, layer i only sees the parameters $\theta_{\mathcal{S}_i} = \mathcal{S}_i(\theta)$, so by the chain rule entry i of $\mathcal{M}^*(h)$[15] is

$$\big(\mathcal{M}^*(h)\big)_i = D_{\theta_{\mathcal{S}_i}}\Phi_i[\mathcal{S}_i(h)] = \sum_{k=1}^{K} \mathcal{M}^*_{ki}(h_k), \quad \mathcal{M}^*_{ki} = D_{\theta_k}\Phi_i, \tag{23}$$

so block (i, k) of $\mathcal{M}^*$ is nonzero only if $\mathcal{S}_i$ selects the parameter block θ_k. The indices of $\mathcal{M}^*_{ki}$ are written in the order ki because it is the adjoint of the block $\mathcal{M}_{ki}$ of $\mathcal{M}$, at position (k, i), as shown below. Written out, with $\mathcal{O}$ denoting the zero map,

$$\mathcal{L} = \begin{pmatrix} \mathcal{I} & \mathcal{O} & \dots & \mathcal{O} \\ \mathcal{L}_{21} & \mathcal{I} & \dots & \mathcal{O} \\ \vdots & \vdots & \ddots & \vdots \\ \mathcal{L}_{N1} & \mathcal{L}_{N2} & \dots & \mathcal{I} \end{pmatrix}, \quad \mathcal{M}^* = \begin{pmatrix} \mathcal{M}^*_{11} & \dots & \mathcal{M}^*_{K1} \\ \vdots & \ddots & \vdots \\ \mathcal{M}^*_{1N} & \dots & \mathcal{M}^*_{KN} \end{pmatrix}. \tag{24}$$

Layer i only reads earlier states, so $\mathcal{L}$ is lower block triangular with identities on the diagonal. In practice a layer reads only a few of the states, and most parameter blocks are used by a single layer, so most blocks are $\mathcal{O}$, and $\mathcal{L}$ and $\mathcal{M}^*$ are sparse. Since the inner products on $\mathcal{U}$ and Θ are sums over the layers and the parameter blocks, the adjoints follow the same pattern as for matrices: transpose the block pattern and take the adjoint of each block, $(\mathcal{L}^*)_{ji} = \mathcal{L}^*_{ij}$.[16] This gives

$$\mathcal{L}^* = \begin{pmatrix} \mathcal{I} & \mathcal{L}^*_{21} & \dots & \mathcal{L}^*_{N1} \\ \mathcal{O} & \mathcal{I} & \dots & \mathcal{L}^*_{N2} \\ \vdots & \vdots & \ddots & \vdots \\ \mathcal{O} & \mathcal{O} & \dots & \mathcal{I} \end{pmatrix}, \quad \mathcal{M} = \begin{pmatrix} \mathcal{M}_{11} & \dots & \mathcal{M}_{1N} \\ \vdots & \ddots & \vdots \\ \mathcal{M}_{K1} & \dots & \mathcal{M}_{KN} \end{pmatrix}. \tag{25}$$

Since $\mathcal{L}^*$ is upper block triangular, the adjoint equation is solved by back substitution from the last layer to the first. Both this solve and the application of $\mathcal{M}$ only require the action of the adjoint blocks on the entries Λ_i.

[14] Recall that $\mathcal{L} : \mathcal{U} \to \mathcal{U}$ maps state changes to state changes, so $\mathcal{L}(E)$ is again a list with one entry per layer.

[15] Recall that $\mathcal{M}^* : \Theta \to \mathcal{U}$ maps parameter changes to the state changes they cause directly: it takes a list over the K parameter blocks to a list over the N layers. Its adjoint $\mathcal{M} : \mathcal{U} \to \Theta$ goes the other way.

[16] This relies on the inner products treating the blocks separately. An inner product that couples the blocks, such as $\langle h, \mathcal{A}(g)\rangle_\Theta$ with the Fisher information $\mathcal{A}$ of natural gradient descent, gives an adjoint that also involves $\mathcal{A}$. In practice one keeps the separable inner product for the derivation and applies $\mathcal{A}^{-1}$ to the resulting gradient afterwards.

Example 4: Affine layers

Example 4 (Affine layers) Let every layer apply an affine map to the states it reads, followed by an entrywise activation σ. Layer i reads the state U_j through a weight W_{ij} and adds a bias b_i, so that

$$\Phi_i(U, \theta) = \sigma(Z_i), \quad Z_i = \sum_j W_{ij}U_j + b_i\mathbf{1}^\top, \quad \Delta_i = \sigma'(Z_i), \tag{26}$$

where the sum runs over the states that layer i reads, and U_0 is the input data. The selection map $\mathcal{S}_i$ picks out the weights W_{ij} and the bias b_i of layer i among the

parameter blocks θ_k, and two layers share a block if, for example, $W_{32} = W_{43}$. A plain feed-forward network only has the weights $W_{i,i-1}$. The product rule and the entrywise linearization of σ give

$$\mathcal{L}_{ij}(E_j) = -\Delta_i \odot (W_{ij} E_j), \quad \mathcal{M}^*_{ki}(h_k) = \begin{cases} \Delta_i \odot (h_k U_j) & \text{if } \theta_k = W_{ij} \\ \Delta_i \odot (h_k \mathbf{1}^\top) & \text{if } \theta_k = b_i \end{cases} \tag{27}$$

where $\odot$ denotes entrywise multiplication.[17] Their adjoints follow from two properties of the Frobenius inner product: entrywise multiplication is self-adjoint, $\langle Z, \Delta \odot X \rangle_F = \langle \Delta \odot Z, X \rangle_F$, and a factor of a matrix product moves to the other side, $\langle Z, AB \rangle_F = \langle A^\top Z, B \rangle_F = \langle ZB^\top, A \rangle_F$.[18] Hence

$$\mathcal{L}^*_{ij}(\Lambda_i) = -W_{ij}^\top (\Delta_i \odot \Lambda_i), \quad \mathcal{M}_{ki}(\Lambda_i) = \begin{cases} (\Delta_i \odot \Lambda_i) U_j^\top & \text{if } \theta_k = W_{ij}. \\ (\Delta_i \odot \Lambda_i) \mathbf{1} & \text{if } \theta_k = b_i \end{cases} \tag{28}$$

[17] The weights W_{i0} read the input data U_0, which is fixed and not part of $\mathcal{U}$, so they give no block in $\mathcal{L}$, but they still give blocks of $\mathcal{M}^*$. Treating U_0 as a state would add the blocks $\mathcal{L}_{i0}$ and give the gradient with respect to the input.

[18] Both equalities follow from $\langle X, Y \rangle_F = \operatorname{tr}(X^\top Y)$. The first only regroups the product,

$$\operatorname{tr}(Z^\top A B) = \operatorname{tr}\Big((A^\top Z)^\top B \Big),$$

and the second uses the cyclic property of the trace,

$$\operatorname{tr}(Z^\top A B) = \operatorname{tr}(B Z^\top A) = \operatorname{tr}\Big((Z B^\top)^\top A \Big).$$

4. A code example

The code is adapted from the Julia notebook Backprop with Backslash accompanying [1], which fixes a block bidiagonal $\mathcal{L}$ and a block diagonal $\mathcal{M}$. Here both are general sparse matrices of operators, so that a layer can read any earlier state and share parameter blocks with other layers.

We introduce a network of $N = 6$ layers, which produce the states $U_1, \dots, U_6$ from the input U_0, and include $m = 10$ data points. The layers are those of Example 4. Besides the chain weights $W_{i,i-1}$, layers 3, 5, and 6 also read U_1, U_2, and U_3 through the skip weights W_{31}, W_{52}, and W_{63}, and layers 3 and 4 share both their chain weight, $W_{32} = W_{43}$, and their bias, $b_3 = b_4$. With 8 weight blocks and 5 bias blocks, $K = 13$.[19] The operator $\mathcal{L}$ then has the pattern

$$\mathcal{L} = \begin{pmatrix} \mathcal{I} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} \\ \mathcal{L}_{2,1} & \mathcal{I} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} \\ \mathcal{L}_{3,1} & \mathcal{L}_{3,2} & \mathcal{I} & \mathcal{O} & \mathcal{O} & \mathcal{O} \\ \mathcal{O} & \mathcal{O} & \mathcal{L}_{4,3} & \mathcal{I} & \mathcal{O} & \mathcal{O} \\ \mathcal{O} & \mathcal{L}_{5,2} & \mathcal{O} & \mathcal{L}_{5,4} & \mathcal{I} & \mathcal{O} \\ \mathcal{O} & \mathcal{O} & \mathcal{L}_{6,3} & \mathcal{O} & \mathcal{L}_{6,5} & \mathcal{I} \end{pmatrix}. \tag{29}$$

[19] Each of the 6 layers has a chain weight and a bias, but layers 3 and 4 share theirs, which leaves $6 - 1 = 5$ pairs. Each of the 3 skip connections adds a weight block of its own, so $K = 2(6-1) + 3 = 13$.

Already here one can see that $\mathcal{L}$ is sparse, with only 14 nonzero blocks out of 36, and only 8 of the 15 blocks below the diagonal. The deeper the network, the sparser it becomes, as each layer reads only a few of the earlier states.

Column i of $\mathcal{M}$ has a block in row k for every parameter block θ_k that $\mathcal{S}_i$ selects.[20] This gives

[20] We number the weights first, $\theta_1 = W_{10}$, $\theta_2 = W_{21}$, $\theta_3 = W_{32} = W_{43}$, $\theta_4 = W_{54}$, and $\theta_5 = W_{65}$ along the chain and $\theta_6 = W_{31}$, $\theta_7 = W_{52}$, and $\theta_8 = W_{63}$ for the skip connections, and the biases $\theta_9, \dots, \theta_{13}$ last, so that, for example, $\mathcal{S}_3(\theta) = (\theta_3, \theta_6, \theta_{11})$ and $\mathcal{S}_4(\theta) = (\theta_3, \theta_{11})$.

$$\mathcal{M} = \begin{pmatrix} \mathcal{M}_{1,1} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} \\ \mathcal{O} & \mathcal{M}_{2,2} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} \\ \mathcal{O} & \mathcal{O} & \mathcal{M}_{3,3} & \mathcal{M}_{3,4} & \mathcal{O} & \mathcal{O} \\ \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{M}_{4,5} & \mathcal{O} \\ \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{M}_{5,6} \\ \mathcal{O} & \mathcal{O} & \mathcal{M}_{6,3} & \mathcal{O} & \mathcal{O} & \mathcal{O} \\ \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{M}_{7,5} & \mathcal{O} \\ \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{M}_{8,6} \\ \mathcal{M}_{9,1} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} \\ \mathcal{O} & \mathcal{M}_{10,2} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} \\ \mathcal{O} & \mathcal{O} & \mathcal{M}_{11,3} & \mathcal{M}_{11,4} & \mathcal{O} & \mathcal{O} \\ \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{M}_{12,5} & \mathcal{O} \\ \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{O} & \mathcal{M}_{13,6} \end{pmatrix}. \tag{30}$$

The rows of the shared weight θ_3 and of the shared bias θ_{11} each hold two blocks, so these two entries of $\mathcal{M}(\Lambda)$ sum the contributions of layers 3 and 4.

In the code, each block is an operator that stores the map and its adjoint, and $\mathcal{L}$ and $\mathcal{M}^*$ are sparse matrices of these operators. The only new methods are the back substitution with $\mathcal{L}^*$ and the application of $\mathcal{M}$, which both visit only the stored blocks. The script prints the patterns of $\mathcal{L}$ and $\mathcal{M}^*$ (the transpose of the pattern of $\mathcal{M}$ above) and compares the gradient with a central difference along a random direction.

julia

backbackback.jl

```julia
using LinearAlgebra, Random, SparseArrays
Random.seed!(42)

# A linear map stored together with its adjoint
struct Operator
    operator
    adjoint
    sym::String
end
Base.adjoint(A::Operator) = Operator(A.adjoint, A.operator, "$(A.sym)′")
Base.:*(A::Operator, X) = A.operator(X)
Base.show(io::IO, A::Operator) = print(io, A.sym)
Base.zero(::Type{Operator}) = Operator(zero, zero, "𝒪") # For printing
const 𝓘 = Operator(identity, identity, "𝓘")

# Adjoints of sparse matrices whose entries are operators.
const AdjointOperators = Adjoint{<:Any, <:SparseMatrixCSC{Operator}}

# Back substitution with ℒ*: Λ_j = g_j - ∑_{i>j} ℒ_ij* Λ_i.
function Base.:\(A::AdjointOperators, g::Vector)
    ℒ = parent(A) # Get the non-adjoint, but apply adjoint later
    rows, blocks = rowvals(ℒ), nonzeros(ℒ) # Column j: blocks[r], rows[r]
    Λ = copy(g)
    for j in length(g):-1:1, r in nzrange(ℒ, j)
        i = rows[r]
        if i > j
            Λ[j] -= blocks[r]' * Λ[i]
        end
    end
    return Λ
end

# ℳ = (ℳ′)*: entry k sums the contributions of the layers that use θ_k.
function Base.:*(A::AdjointOperators, Λ::Vector)
    ℳ′ = parent(A) # Get the non-adjoint, but apply adjoint later
    rows, blocks = rowvals(ℳ′), nonzeros(ℳ′)
```

```
    return [sum(blocks[r]' * Λ[rows[r]] for r in nzrange(𝛭′, k))
            for k in axes(𝛭′, 2)]
end

# Defining the activation function and its derivative
σ(x) = exp(-x)
σ′(x) = -exp(-x)

# Layer i reads the state U_j through the weight W_ij = θ_k for every
# j => k in W[i], and adds the bias b_i = θ_b[i]. Layers 3 and 4 share
# both their weight θ₃ and their bias θ₁₁.
d = [5, 4, 4, 4, 4, 3, 1] # Feature sizes of U₀, …, U₆
W = [[0 => 1], [1 => 2], [2 => 3, 1 => 6], [3 => 3], [4 => 4, 2 => 7],
     [5 => 5, 3 => 8]]
b = [9, 10, 11, 11, 12, 13]
N, K = 6, 13 # Layers, parameter blocks

function network(θ)
    U, Δ = Matrix{Float64}[], Matrix{Float64}[]
    state(j) = j == 0 ? U₀ : U[j]
    for i in 1:N
        Z = sum(θ[k] * state(j) for (j, k) in W[i]) .+ θ[b[i]]
        push!(U, σ.(Z))
        push!(Δ, σ′.(Z))
    end
    return U, Δ
end

# The sparse matrices 𝓛 = I - D_U Φ and 𝛭′ = D_θ Φ, block by block.
function operators(θ, U, Δ)
    state(j) = j == 0 ? U₀ : U[j]
    𝓛, 𝛭′ = spzeros(Operator, N, N), spzeros(Operator, N, K)
    for i in 1:N
        𝓛[i, i] = 𝓘
        for (j, k) in W[i]
            Wij, Uj = θ[k], state(j)
            if j > 0
                𝓛[i, j] = Operator(E -> -(Δ[i] .* (Wij * E)),
                                   Λ -> -(Wij' * (Δ[i] .* Λ)), "𝓛")
            end
            𝛭′[i, k] = Operator(h -> Δ[i] .* (h * Uj),
                               Λ -> (Δ[i] .* Λ) * Uj', "𝛭′")
        end
        𝛭′[i, b[i]] = Operator(h -> Δ[i] .* h,
                              Λ -> vec(sum(Δ[i] .* Λ, dims=2)), "𝛭′")
    end
    return 𝓛, 𝛭′
end

# The data and the parameters.
m = 10 # Data points
α = 1e-2 # Weight of the regularization α/2 ⟨θ, θ⟩
init(dims...) = 0.01 * randn(dims...)
U₀, Y = init(d[1], m), init(d[end], m)
θ = Vector{Array{Float64}}(undef, K)
for i in 1:N
    for (j, k) in W[i]
        θ[k] = init(d[i+1], d[j+1])
    end
    θ[b[i]] = init(d[i+1])
end

# The gradient of J(θ) = ‖U_N - Y‖²/(2m) + α/2 ⟨θ, θ⟩.
J(θ) = norm(network(θ)[1][N] - Y)^2 / (2m) + α / 2 * dot(θ, θ)
U, Δ = network(θ)
𝓛, 𝛭′ = operators(θ, U, Δ)
∇Uℓ = [i == N ? (U[N] - Y) / m : zero(U[i]) for i in 1:N] # 𝒫*(∇f)
Λ = 𝓛' \ ∇Uℓ
∇J = 𝛭′' * Λ + α * θ # 𝛭(Λ) + αθ
display(𝓛)
```

```
display(𝓜′)

# Central difference of the loss along a random direction.
h = [randn(size(p)...) for p in θ]
ε = 1e-6
directional = (J(θ + ε * h) - J(θ - ε * h)) / (2ε)
versus_finite_difference = abs(directional - dot(∇J, h)) / abs(dot(∇J, h))
@show versus_finite_difference
```

5. Conclusion

Going from a single input to a batch only changed the entries of the matrices, from scalars to linear maps. The overall structure stayed the same: $\mathcal{L}$ is a unit lower block triangular matrix whose sparsity pattern is a DAG, and both $\mathcal{L}$ and $\mathcal{M}$ are most often sparse. The formula for the gradient also kept the same form, $\nabla_\theta \mathcal{J} = \mathcal{M}\left((\mathcal{L}^*)^{-1}\nabla_U \ell\right) + \nabla_\theta \ell$,[21] and can still be computed with a single back substitution and a single sparse product.

[21] In the previous post the gradient reads $\nabla_\theta \mathcal{J} = ML^{-\top}P^\top \nabla_{u_P}\ell + \mu\nabla_\theta \mathcal{R}$. The matrices M and L become the operators $\mathcal{M}$ and $\mathcal{L}$, the transpose becomes the adjoint, $P^\top \nabla_{u_P}\ell$ becomes $\nabla_U \ell = \mathcal{P}^*(\nabla f)$, and the regularization is part of $\nabla_\theta \ell$.

Bibliography

[1] A. Edelman, E. Akyürek, and Y. Wang, "Backpropagation through Back Substitution with a Backslash," *SIAM Journal on Matrix Analysis and Applications*, vol. 45, no. 1, pp. 429–449, 2024, doi: 10.1137/22M1532871.

[2] P. Bright, A. Edelman, and S. G. Johnson, "Matrix Calculus (for Machine Learning and Beyond)." [Online]. Available: https://arxiv.org/abs/2501.14787